CI/CD Pipeline Architecture

Confidential Enterprise Infrastructure Project · Bitbucket Pipelines with OIDC federation to AWS — no long-lived access keys

Nisal G.
AWS Cloud Infrastructure & DevOps Engineer

A · INFRASTRUCTURE PIPELINE (TERRAFORM)

IMPLEMENTED Defined in the infrastructure repository's *bitbucket-pipelines.yml* · pinned Terraform image · 30-minute step limit

On every pull request

Pull request
Feature branch → PR to main. Pipeline runs automatically.

→ **OIDC → plan role**
Short-lived credentials. ReadOnlyAccess + state-lock object only.

→ **fmt -check · init**
Formatting gate. S3 backend, encrypted state, native lock file.

→ **terraform validate**
Configuration and variable validation rules checked.

→ **terraform plan**
Saved plan (tfplan) + plan.txt published as artifacts for review.

→ **Human review**
Team rule: read the plan (log or plan.txt) before merging.

After merge to protected main (PR-only, passing build required)

Merge to main
Branch protection: pull requests only, build must pass.

→ **Plan step**
fmt · init · validate · plan with the read-only role.

→ **MANUAL APPLY GATE**
trigger: manual — nothing changes in AWS until an engineer clicks Run.

→ **OIDC → apply role**
Assumable only from the 'dev' deployment environment (sub claim).

→ **terraform apply tfplan**
Applies exactly the reviewed, saved plan — no re-plan drift.

→ **AWS workload account**
Dev environment updated. Rule: no console changes, Git only.

B · APPLICATION PIPELINE (BACKEND CONTAINERS)

IMPLEMENTED Dockerfile, *bitbucket-pipelines.yml* and ECS deploy script in the backend repository · IAM roles defined in Terraform

Merge to development
Branch trigger. Also runnable by hand on any branch (custom).

→ **Multi-stage build**
BuildKit cross-compiles to ARM64; base images pinned by digest.

→ **Distroless runtime**
Static binary, no shell, non-root. .dockerignore keeps .env out.

→ **OIDC → push to ECR**
Push-only role. Tag = build number (immutable tags, scan on push).

→ **OIDC → deploy role**
Only from the 'dev' deployment. Checks the database is available.

→ **Migrate → roll out**
New task defs → migrations (exit 0) → update → wait → fail on rollback.

Security: deploy and apply roles trust only OIDC tokens whose *sub* claim names the repository *and* its 'dev' deployment environment; build and plan steps get narrower roles. No AWS keys stored anywhere.

DOCUMENTED Admin web pipeline (separate repo): lint · typecheck · tests · Trivy · SBOM → ECR → same deploy pattern. · Static website: S3 sync + CloudFront invalidation through its own scoped role.

C · QUALITY GATES & PROMOTION PATH

PLANNED Designed in the architecture document or noted in the pipeline file; not yet configured.

Backend test gate in CI
Re-enable PR pipeline (make check: lint, gosec, vet, tests). Runs locally today.

→ **Build once, promote**
Same image promoted across accounts (shared registry).

→ **Staging gate**
QA approval in the pipeline before the staging deploy.

→ **Production gate**
Two approvals · deploy-freeze window.

→ **Canary release**
Canary 10% → 100% with alarm-driven rollback.

Drift detection
Scheduled plan -detailed-exitcode + alert.